Bases de la programmation

Épreuve écrite — Sujet A

Durée : 1 h 30 • Examen blanc (entraînement)

Consignes. Documents, calculatrices et téléphones interdits. Les programmes sont à écrire en Python, sur la copie, avec une indentation lisible. Les seules opérations autorisées sur les listes sont la création d'une liste vide `[]`, `len(L)`, l'accès `L[i]`, `L.append(x)` et le parcours `for x in L`. Sont interdits : l'opérateur `in` (en dehors de `for`), les méthodes `sort`, `count`, `index`, et les fonctions `sorted`, `min`, `max`, `sum`. Une fonction peut toujours utiliser les fonctions écrites aux questions précédentes, même si vous ne les avez pas traitées. Le barème est donné à titre indicatif (total sur 20).

Exercice 1 — Lecture de code *4 points*

Pour chacun des programmes suivants, indiquer *exactement* ce qui s'affiche à l'écran.

a)

```
a = 17
b = 5
print(a // b, a % b)
a, b = b, a % b
print(a, b)
```

b)

```
s = 0
i = 1
while i < 10:
    s = s + i
    i = i + 3
print(i, s)
```

c) Indiquer ce qui s'affiche, puis expliquer en une phrase ce que fait ce programme pour une liste `L` quelconque.

```
L = [2, 2, 3, 3, 3, 2, 5, 5]
R = []
for x in L:
    if len(R) == 0 or x != R[len(R) - 1]:
        R.append(x)
print(R, len(R))
```

d) Indiquer ce qui s'affiche, puis expliquer en une ou deux phrases pourquoi `T` est modifiée alors que `y` ne l'est pas.

```
def f(L, x):
    L[0] = L[0] + x
    x = x * 2
    return x

T = [1, 2]
y = 3
```

```
z = f(T, y)
print(T, y, z)
```

Exercice 2 — Tarif de cinéma *3 points*

Un cinéma applique les tarifs suivants selon l'âge du spectateur : moins de 14 ans : 5 € ; de 14 à 25 ans inclus : 7 € ; plus de 65 ans (strictement) : 6 € ; dans tous les autres cas : 10 €. Le mercredi, tous les tarifs baissent de 2 €, *sauf* le tarif enfant (moins de 14 ans).

Écrire un programme qui demande l'âge du spectateur (un entier) puis le jour de la semaine (une chaîne de caractères, par exemple `"mercredi"`), et qui affiche le prix à payer. Si l'âge saisi est négatif, le programme affiche « Âge incorrect » et ne calcule rien.

```
Âge : 30
Jour : mercredi
Prix à payer : 8 euros
```

Exercice 3 — Boucles *3 points*

1. Écrire un programme qui saisit une suite de notes réelles. La saisie s'arrête lorsque l'utilisateur tape `-1`. Toute autre valeur hors de l'intervalle $[0, 20]$ est refusée et redemandée. À la fin, le programme affiche le nombre de notes saisies, leur moyenne et le nombre de notes supérieures ou égales à 10. Si aucune note n'a été saisie, il affiche « Aucune note ».

2. Écrire un programme qui demande une hauteur `h` (entier strictement positif, saisie contrôlée) et qui dessine un triangle d'étoiles aligné à droite. Exemple pour `h = 4` :

```
   *
  **
 ***
****
```

Exercice 4 — Déficient, parfait ou abondant *3 points*

On appelle *diviseur propre* d'un entier $n \geq 1$ tout diviseur de n strictement inférieur à n (1 compris). Un entier est dit *déficient* si la somme de ses diviseurs propres est strictement inférieure à lui-même, *parfait* si elle lui est égale, et *abondant* si elle lui est strictement supérieure. Exemples : 8 est déficient ($1 + 2 + 4 = 7$), 6 est parfait ($1 + 2 + 3 = 6$), 12 est abondant ($1 + 2 + 3 + 4 + 6 = 16$).

1. Écrire une fonction `somme_diviseurs(n)` qui renvoie la somme des diviseurs propres de `n`.

2. Écrire une fonction `nature(n)` qui renvoie la chaîne `"deficient"`, `"parfait"` ou `"abondant"`.

3. Écrire une fonction `compter(L)` qui reçoit une liste d'entiers et renvoie *trois* valeurs : le nombre de déficients, de parfaits et d'abondants de la liste.

```
compter([8, 6, 12, 28, 7])     # 2, 2, 1
```

Exercice 5 — Listes *4 points*

1. L'opérateur `in` étant interdit, écrire une fonction `appartient(L, x)` qui renvoie `True` si `x` est un élément de `L`, `False` sinon.

2. Écrire une fonction `sans_doublons(L)` qui renvoie une *nouvelle* liste contenant les valeurs de `L` une seule fois chacune, dans l'ordre de leur première apparition.

```
sans_doublons([4, 1, 4, 2, 1, 4])     # [4, 1, 2]
sans_doublons([])                     # []
```

3. Écrire une fonction `intersection(L1, L2)` qui renvoie la liste, sans doublons, des valeurs présentes à la fois dans `L1` et dans `L2`.

```
intersection([1, 2, 2, 5, 7], [7, 2, 9, 2])     # [2, 7]
```

4. Écrire une fonction `plus_longue_montee(L)` qui renvoie la longueur de la plus longue suite d'éléments *consécutifs* de `L` strictement croissante (0 si la liste est vide).

```
plus_longue_montee([3, 1, 2, 5, 4, 6])     # 3  (1, 2, 5)
plus_longue_montee([5, 4, 3])              # 1
```

Exercice 6 — Matrices *3 points*

Une matrice est représentée par une liste de listes ; comme en TD, toute fonction qui reçoit une matrice `M` reçoit aussi son nombre de lignes `n` et son nombre de colonnes `m`. Exemple :

```
M = [[2, 7, 6],
     [9, 5, 1],
     [4, 3, 8]]
```

1. Écrire une fonction `somme_ligne(M, n, m, i)` qui renvoie la somme des éléments de la ligne d'indice `i`, et une fonction `somme_colonne(M, n, m, j)` qui renvoie la somme des éléments de la colonne d'indice `j`.

2. Une matrice carrée est un *carré magique* si les sommes de chacune de ses lignes, de chacune de ses colonnes et de ses deux diagonales sont toutes égales (la matrice `M` ci-dessus est un carré magique : toutes les sommes valent 15). Écrire une fonction `est_magique(M, n, m)` qui renvoie `True` si `M` est un carré magique, `False` sinon (une matrice non carrée n'est pas magique).

Bases de la programmation

Épreuve écrite — Sujet B

Durée : 1 h 30 • Examen blanc (entraînement)

Consignes. Documents, calculatrices et téléphones interdits. Les programmes sont à écrire en Python, sur la copie, avec une indentation lisible. Les seules opérations autorisées sur les listes sont la création d'une liste vide `[]`, `len(L)`, l'accès `L[i]`, `L.append(x)` et le parcours `for x in L`. Sont interdits : l'opérateur `in` (en dehors de `for`), les méthodes `sort`, `count`, `index`, `insert`, et les fonctions `sorted`, `min`, `max`, `sum`. Une fonction peut toujours utiliser les fonctions écrites aux questions précédentes, même si vous ne les avez pas traitées. Le barème est donné à titre indicatif (total sur 20).

Exercice 1 — Lecture de code *4 points*

Pour chacun des programmes suivants, indiquer *exactement* ce qui s'affiche à l'écran.

a)

```
x = 7
if x > 5 and x % 2 == 0:
    print("A")
elif x > 5 or x < 0:
    print("B")
else:
    print("C")
print(x // 2 * 2 == x)
```

b) Indiquer ce qui s'affiche, puis dire en une phrase ce que représentent `c` et `p` à la fin, pour une liste `L` et une valeur `v` quelconques.

```
L = [4, 7, 2, 7, 1]
v = 7
c = 0
p = -1
for i in range(len(L)):
    if L[i] == v:
        c = c + 1
        if p == -1:
            p = i
print(c, p)
```

c)

```
for i in range(3):
    for j in range(i, 3):
        print(i + j, end=" ")
    print()
```

d)

```
M = [[1, 2, 3], [4, 5, 6]]
s = 0
for i in range(2):
```

```
    s = s + M[i][2 - i]
print(s, M[1][0] + len(M) + len(M[0]))
```

Exercice 2 — Nature d'un triangle *3 points*

Écrire un programme qui demande trois longueurs réelles `a`, `b` et `c`, puis affiche :

- « Longueurs incorrectes » si l'une des longueurs est négative ou nulle ;
- sinon « Triangle impossible » si l'une des longueurs est supérieure ou égale à la somme des deux autres ;
- sinon « Équilatéral », « Isocèle » ou « Quelconque » selon le nombre de côtés égaux.

Exercice 3 — Boucles *4 points*

1. **PGCD.** L'algorithme d'Euclide calcule le plus grand commun diviseur de deux entiers strictement positifs a et b : tant que $b \neq 0$, on remplace le couple (a, b) par $(b,\ a \bmod b)$; à la fin, le PGCD est la valeur de a. Écrire un programme qui saisit deux entiers strictement positifs (saisie contrôlée pour chacun) et affiche leur PGCD. Exemple : pour 84 et 36, le programme affiche 12.

2. **Suite de Syracuse.** Partant d'un entier $n > 0$, on construit une suite : si le terme courant est pair, on le divise par 2 ; sinon, on le multiplie par 3 et on ajoute 1. On s'arrête lorsque l'on atteint 1. Écrire un programme qui saisit n (saisie contrôlée), affiche tous les termes de la suite sur une même ligne, puis le nombre d'étapes nécessaires pour atteindre 1 et la plus grande valeur atteinte.

```
n = 6
6 3 10 5 16 8 4 2 1
Nombre d'étapes : 8
Valeur maximale : 16
```

Exercice 4 — Nombres palindromes *4 points*

Un entier positif est un *palindrome* s'il se lit de la même façon de gauche à droite et de droite à gauche (exemples : 7, 44, 1221, 12321).

1. Écrire une fonction `nb_chiffres(n)` qui renvoie le nombre de chiffres de l'entier positif `n` (on considère que 0 a un chiffre).

2. Écrire une fonction `inverse(n)` qui renvoie l'entier obtenu en lisant les chiffres de `n` à l'envers. Exemple : `inverse(1230)` renvoie 321. Les chaînes de caractères sont interdites dans cette question.

3. Écrire une fonction `est_palindrome(n)` qui renvoie un booléen.

4. Écrire une fonction `compter_palindromes(a, b)` qui renvoie le nombre de palindromes compris entre `a` et `b` (inclus), les bornes pouvant être données dans n'importe quel ordre.

Exercice 5 — Listes triées *5 points*

1. Écrire une fonction `sens(L)` qui renvoie `1` si la liste est croissante (au sens large : chaque élément est supérieur ou égal au précédent), `-1` si elle est décroissante au sens large et pas croissante, et `0` sinon.

```
sens([1, 2, 2, 5])      # 1
sens([9, 4, 4, 1])      # -1
sens([3, 1, 2])         # 0
sens([7])               # 1
```

2. Écrire une fonction `inserer(L, x)` qui insère la valeur `x` à sa place dans une liste `L` déjà triée par ordre croissant, de sorte que `L` reste triée (la fonction modifie `L` et ne renvoie rien). On ajoutera d'abord `x` à la fin avec `append`, puis on le fera « redescendre » par échanges successifs avec son voisin de gauche tant que celui-ci est plus grand.

3. En déduire une fonction `tri_insertion(L)` qui renvoie une *nouvelle* liste contenant les éléments de `L` triés par ordre croissant, construite en insérant un à un les éléments de `L` dans une liste initialement vide. La liste `L` ne doit pas être modifiée.

4. Pour `L = [5, 3, 8, 1, 4]`, donner le contenu de la nouvelle liste après chacune des insertions effectuées par `tri_insertion`.

5. Écrire une fonction `difference(L1, L2)` qui prend deux listes triées par ordre croissant et *sans doublons*, et renvoie la liste triée des éléments de `L1` qui ne sont pas dans `L2`. On ne fera qu'*un seul parcours simultané* des deux listes (comme pour une fusion), sans boucles imbriquées.

```
difference([1, 3, 4, 7, 9], [2, 3, 7, 8])     # [1, 4, 9]
difference([1, 2], [])                        # [1, 2]
```